```
# eagleeye.py
# Real-time observer and corrector module for Lucy 3.0.
# Integrates deeply across the stack: hooks into Local Agents (per-step observation),
# Orchestrator (pre/post-dispatch monitoring), SystemGuardian (feeds detections for
veto/resolution),
# Dashboard (pushes live confidence/alerts), DejaVu (long-loop pattern input),
# and Safe Execution Gate (pre-execution confidence gates).
# Supports parallelism: multiple concurrent monitors for agent swarms.
# Loops: Short (real-time corrective pulses), Medium (session anomaly aggregation),
# Long (historical drift detection via DejaVu integration).

import asyncio
from typing import Dict, Any, List
from confidence_monitor import ConfidenceMonitor  # Sub-module for scoring
from dashboard import Dashboard  # For live feeds
from system_guardian import SystemGuardian  # For escalations
from dejavu_agent import DejaVuCatcher  # For long-loop patterns
from safe_execute import safe_execute  # Optional: if corrective needs safe re-exec

class EagleEyeWatcher:
    def __init__(self):
        self.monitors: Dict[str, asyncio.Task] = {}  # agent_id/subtask_id -> monitor task
        self.confidence_monitor = ConfidenceMonitor()
        self.active_observations: Dict[str, List[Observation]] = {}  # Per-agent history (medium loop
buffer)
        self.anomaly_threshold = 0.7  # Tune: confidence below this → correct/escalate
        self.loop = asyncio.get_event_loop()

    async def observe_step(self, agent_id: str, step: Step) -> Observation:
        """
        Core per-step observation: called from AgentDispatcher._run_single_agent.
        - Analyzes step for errors/anomalies/self-consistency.
        - Computes confidence.
        - Provides corrective feedback if needed.
        - Feeds live to Dashboard; aggregates for medium loops.
        """
        # 1. Self-observation: inspect agent's internal state/output
        state_snapshot = await self._capture_state(agent_id, step)  # e.g., memory dump, output
preview

        # 2. Error detection: heuristic/ML-based checks
        errors = self._detect_errors(state_snapshot, step)  # e.g., scope violations, logic
inconsistencies
```

```
        # 3. Confidence scoring (parallel if multi-metrics)
        confidence_task =
self.loop.create_task(self.confidence_monitor.compute_confidence(step, state_snapshot))
        confidence = await confidence_task

        # 4. Corrective feedback generation
        feedback = None
        requires_correction = confidence < self.anomaly_threshold or errors
        if requires_correction:
            feedback = await self._generate_corrective_feedback(errors, confidence, step)
            # Short loop: push correction immediately
            await self._apply_short_loop_correction(agent_id, feedback)

        # 5. Record observation (for medium/long loops)
        obs = Observation(agent_id=agent_id, step=step, errors=errors, confidence=confidence,
feedback=feedback)
        self._record_observation(agent_id, obs)

        # 6. Live push to Dashboard (async, non-blocking)
        self.loop.create_task(Dashboard.push_live_alert(obs))  # Confidence + any alerts

        # 7. Feed to Guardian for potential veto (if critical)
        if requires_correction and self._is_critical(obs):
            await SystemGuardian.notify_anomaly(obs)  # Could trigger runtime veto

        return obs

    async def monitor_agent(self, agent_id: str):
        """
        Background full-agent monitor: called from AgentDispatcher._eagle_eye_monitor.
        Runs continuously while agent active; checks for drift over time.
        """
        while agent_id in self.active_agents:  # Assume shared active_agents or query Dispatcher
            # Medium loop: aggregate recent observations
            recent_obs = self.active_observations.get(agent_id, [])[-10:]  # Last 10 for pattern
            anomalies = self._detect_drift(recent_obs)  # e.g., declining confidence trend

            if anomalies:
                # Escalate to Guardian
                await SystemGuardian.notify_drift(agent_id, anomalies)
                # Long loop: push pattern to DejaVu
                DejaVuCatcher.record_pattern(anomalies)

            # Sleep for monitoring interval (tune for low-latency)
```

```
            await asyncio.sleep(1.0)

    async def pre_dispatch_monitor(self, task: Task):
        """
        Integration with Orchestrator: pre-flight observation.
        Called from Orchestrator._pre_flight_single (expand that if needed).
        Early confidence estimate on task feasibility.
        """
        initial_confidence = await self.confidence_monitor.estimate_task_confidence(task)
        if initial_confidence < 0.5:  # Early flag
            return Observation(task_id=task.id, confidence=initial_confidence, feedback="Low initial
confidence; refine task?")
        return None  # No issue

    async def post_execution_review(self, result: Result):
        """
        Integration with Safe Execution Gate: post-exec hook.
        Called after safe_execute to verify outcome vs. expectation.
        """
        expected = result.expected_outcome  # From plan
        actual = result.actual_outcome
        discrepancy = self._compute_discrepancy(expected, actual)
        if discrepancy > 0.3:  # Threshold
            obs = Observation(result_id=result.id, errors=[f"Discrepancy: {discrepancy}"],
confidence=1 - discrepancy)
            self._record_observation(result.agent_id, obs)
            # Feed back to medium/long loops
            await SystemGuardian.notify_post_exec(obs)
            DejaVuCatcher.record_pattern(obs)

    # Helper methods
    async def _capture_state(self, agent_id: str, step: Step) -> Dict:
        # Mock: in real, hook into agent internals (e.g., via callbacks or shared memory)
        return {"memory": "snapshot", "output_preview": step.preview}

    def _detect_errors(self, state: Dict, step: Step) -> List[str]:
        # Heuristics: e.g., if step.action == "delete_file" and not in scope, flag
        errors = []
        if "out_of_scope" in state:
            errors.append("Scope violation")
        return errors

    async def _generate_corrective_feedback(self, errors: List, confidence: float, step: Step) ->
Dict:
```

```
        # Simple: could use small LLM for suggestions
        return {"suggestion": "Adjust plan: avoid X due to Y", "revised_step":
step.with_correction(errors)}

    async def _apply_short_loop_correction(self, agent_id: str, feedback: Dict):
        # Push back to agent (assume agent has receive_feedback method)
        agent = get_agent_by_id(agent_id)  # From Dispatcher or global registry
        await agent.receive_feedback(feedback)

    def _record_observation(self, agent_id: str, obs: Observation):
        if agent_id not in self.active_observations:
            self.active_observations[agent_id] = []
        self.active_observations[agent_id].append(obs)
        # Prune if too long (medium loop buffer)

    def _detect_drift(self, observations: List[Observation]) -> List[Anomaly]:
        # Trend analysis: e.g., if confidence decreasing linearly
        confidences = [o.confidence for o in observations]
        if len(confidences) > 3 and all(confidences[i] > confidences[i+1] for i in
range(len(confidences)-1)):
            return [Anomaly(type="declining_confidence", severity="medium")]
        return []

    def _is_critical(self, obs: Observation) -> bool:
        # e.g., if "security" in obs.errors
        return any("critical" in e.lower() for e in obs.errors)

    def _compute_discrepancy(self, expected: Any, actual: Any) -> float:
        # Simple diff: could be string similarity, etc.
        return 0.0 if expected == actual else 0.5  # Placeholder

# Usage example (from agent_dispatch.py or orchestrator.py)
# eagle_eye = EagleEyeWatcher()
# obs = await eagle_eye.observe_step("agent_123", current_step)
# await eagle_eye.monitor_agent("agent_123")  # As background task
```